Name: A.Sai Ram Prasad

Reg: 1925260187

Course: CSA0824-PYTHON Progamming

Github link: https://github.com/sairamprasad2008-pixel/CSA0824-Python-Program-

TASK 1 – Understand the Problem and Identify the Data

Aim:

To identify, classify, and mathematically define all inputs, constants, computational variables, and physical invariants of the village milk billing pipeline, establishing robust domain validation rules and contrasting scalable aggregate structures against fragmented scalar variables.

Theory:

A milk billing system converts the milk supplied by farmers into accurate payments. It records the farmer ID, milk quantity, fat percentage, SNF percentage, and collection shift. Fixed values such as the base rate and quality standards should be stored as named constants. The system must validate all inputs before calculating payment. Invalid quantities or unrealistic milkquality values should be rejected. Using lists or structured records makes the program easier to manage as the number of farmers increases.

PROGRAM

```python

# Milk Farmer Payment System

# Named constants
(constant) EXTRA_FAT_RATE: float = 2
EXTRA_FAT_RATE = 2.0
STANDARD_FAT = 3.0
MIN_FAT = 2.0
MAX_FAT = 7.0
DAYS = 10

# Inputs
farmer_id = input("Enter Farmer ID: ")
litres = float(input("Enter milk quantity in litres: "))
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))
shift = input("Enter shift (Morning/Evening): ").strip().title()

# Validation
if litres <= 0:
    print("Error: Litres must be greater than zero.")
elif fat < MIN_FAT or fat > MAX_FAT:
    print("Error: Fat percentage must be between 2% and 7%.")
elif snf < 0 or snf > 100:
    print("Error: SNF percentage must be between 0 and 100.")
elif shift not in ["Morning", "Evening"]:
    print("Error: Enter Morning or Evening.")
else:
    # Payment calculation
    extra_fat = max(0, fat - STANDARD_FAT)
    rate = BASE_RATE + (extra_fat * EXTRA_FAT_RATE)
    amount = litres * rate

    # Assumption: same quantity and rate for 10 days
    ten_day_total = amount * DAYS

    # Outputs
    print("\n--- Farmer Payment Details ---")
    print("Farmer ID:", farmer_id)
    print("Milk Quantity:", litres, "litres")
    print("Fat Percentage:", fat, "%")
    print("SNF Percentage:", snf, "%")
    print("Shift:", shift)
    print("Rate per Litre: Rs.", round(rate, 2))
    print("Amount Payable: Rs.", round(amount, 2))
    print("Ten-Day Total: Rs.", round(ten_day_total, 2))
```

INPUT

```
Enter Farmer ID: F101
Enter milk quantity in litres: 10
Enter fat percentage: 5
Enter SNF percentage: 8.5
Enter shift (Morning/Evening): Morning
```

OUTPUT

```
Farmer ID: F101
Milk Quantity: 10.0 litres
Fat Percentage: 5.0 %
SNF Percentage: 8.5 %
Shift: Morning
Rate per Litre: Rs. 34.0
Amount Payable: Rs. 340.0
Ten-Day Total: Rs. 3400.0
```

TASK 2 – Explore Values and Types in the Interpreter Aim:

To analyze CPython’s runtime type system through interactive execution, examine the architectural limitations of IEEE 754 binary floating-point representations in financial arithmetic, and validate high-precision alternatives including the decimal module and integer paise.

Theory:

Python supports data types such as integers, floating-point numbers, and strings. Input values must use the correct type for calculations. Floating-point numbers can produce small rounding errors because some decimal values cannot be represented exactly in binary. Such errors may affect the total payment when many transactions are processed. The Decimal type or integer paise can help maintain accurate financial calculations. Choosing suitable data types is important for reliable billing.

PROGRAM

```python

from decimal import Decimal
import sys

# Explore data types
print("--- Data Types ---")
print(type(10))
print(type(10.0))
print(type("10"))
print(type(True))
print(type([10, 8.5]))

# Explore division operators
print("\n--- Division Operators ---")
print("7 / 2 =", 7 / 2)
print("7 // 2 =", 7 // 2)
print("7 % 2 =", 7 % 2)
print("-7 // 2 =", -7 // 2)

# Floating-point precision
print("\n--- Floating-Point Issue ---")
print("0.1 + 0.2 =", 0.1 + 0.2)
print("0.1 + 0.2 == 0.3:", 0.1 + 0.2 == 0.3)

# Payment amounts stored as strings
payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
```

```
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)  # TODO 1 completed
    return total

print("\n--- Payment Totals ---")
print("float:", total_float(payments))
print("Decimal:", total_decimal(payments))
print("Decimal comparison:",
      Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))

# Compare Decimal(amount) with Decimal(float(amount))
print("\n--- Decimal Conversion ---")
print('Decimal("0.10") =', Decimal("0.10"))
print("Decimal(float('0.10')) =", Decimal(float("0.10")))

# Memory used by one representative value
print("\n--- Memory Usage ---")
print("Integer 10:", sys.getsizeof(10), "bytes")
print("Float 10.0:", sys.getsizeof(10.0), "bytes")
print("Decimal('10.00'):", sys.getsizeof(Decimal("10.00")), "bytes")
```

OUTPUT

```
--- Data Types ---
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'list'>

--- Division Operators ---
7 / 2 = 3.5
7 // 2 = 3
7 % 2 = 1
-7 // 2 = -4

--- Floating-Point Issue ---
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3: False

--- Payment Totals ---
float: 33400.30000000061
Decimal: 33400.30
Decimal comparison: True

--- Decimal Conversion ---
Decimal("0.10") = 0.10
Decimal(float('0.10')) = 0.1000000000000000055511151231257827021181583404541015625

--- Memory Usage ---
Integer 10: 28 bytes
Float 10.0: 24 bytes
Decimal('10.00'): 120 bytes
```

–

TASK 3 Choose Variables and Name the Constants

Aim:

To establish clean symbolic naming conventions following PEP 8 guidelines, analyze variable binding dynamics within CPython’s memory model, demonstrate atomic variable swapping, and evaluate single-point maintenance models against inlined magic numbers and dictionary rate structures.

Theory:

Clear variable names make a program easier to understand and maintain. Python commonly uses snake_case for variables and UPPER_SNAKE_CASE for constants. Constants are used for values such as the base milk rate and quality limits. Python does not strictly prevent a constant from being changed, so uppercase naming acts as a convention. Variables refer to objects and can be assigned new values during execution. Keeping rates in one place reduces repeated code and makes future updates easier.

```python
# Named constants
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30
MIN_FAT = 2.0
MAX_FAT = 7.0
MAX_LITRES = 1000.0

# Rate card
RATE_CARD = {
    "base": BASE_RATE,
    "fat": FAT_RATE,
    "snf": SNF_RATE
}

# Get input
farmer_id = input("Enter Farmer ID: ")
litres = float(input("Enter milk quantity in litres: "))
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))

# Validate and calculate
if litres <= 0 or litres > MAX_LITRES:
    print("Invalid milk quantity.")
elif fat < MIN_FAT or fat > MAX_FAT:
    print("Invalid fat percentage.")
elif snf < 0 or snf > 100:
    print("Invalid SNF percentage.")
else:
    rate = (RATE_CARD["base"]
            + fat * RATE_CARD["fat"]
```

```
            + snf * RATE_CARD["snf"])

    amount = litres * rate

    print("\n--- Farmer Payment Details ---")
    print("Farmer ID:", farmer_id)
    print("Rate per litre: Rs.", round(rate, 2))
    print("Amount payable: Rs.", round(amount, 2))

# Dynamic typing
value = 10
print("\nValue:", value, "| Type:", type(value).__name__)

value = "Morning"
print("Value:", value, "| Type:", type(value).__name__)

# Multiple assignment and swapping
a, b = 10, 20
print("\nBefore swapping:", a, b)

a, b = b, a
print("After swapping:", a, b)
```

OUTPUT

```
Enter Farmer ID: F101
Enter milk quantity in litres: 10
Enter fat percentage: 5
Enter SNF percentage: 8.5
```

INPUT

```
--- Farmer Payment Details ---
Farmer ID: F101
Rate per litre: Rs. 34.55
Amount payable: Rs. 345.5

Value: 10 | Type: int
Value: Morning | Type: str

Before swapping: 10 20
After swapping: 20 10
```

TASK 4 Write Correct Expressions

Aim:

To formulate robust pricing expressions incorporating operator precedence and associativity, resolve floating-point truncation anomalies using proper rounding techniques, contrast round(), int(), and math.floor(), and validate logical expressions using chained comparisons.

Theory:

Python follows operator-precedence rules when evaluating mathematical expressions. Parentheses can be used to make billing formulas clearer. Milk quality premiums or penalties are calculated by comparing fat and SNF values with their standard levels. Floating-point approximations may create values slightly above or below an expected whole number. Using appropriate rounding helps avoid incorrect quality-step calculations. Chained comparisons provide a simple way to check whether values fall within valid limits.

PROGRAM

```python
import math

# Named constants
BASE_RATE = 30.0
FAT_STEP_RATE = 0.40
SNF_STEP_RATE = 0.30
STEP_SIZE = 0.1

# Input
fat = float(input("Enter fat percentage: "))
snf = float(input("Enter SNF percentage: "))
litres = float(input("Enter milk quantity in litres: "))

# Calculate steps using round()
fat_steps = round((fat - 3.0) / STEP_SIZE)
snf_steps = round((snf - 8.0) / STEP_SIZE)

# Calculate rate and payment
rate = BASE_RATE + fat_steps * FAT_STEP_RATE + snf_steps * SNF_STEP_RATE
amount = litres * rate

print("\n--- Payment Calculation ---")
print("Fat steps:", fat_steps)
print("SNF steps:", snf_steps)
print("Rate per litre: Rs.", round(rate, 2))
print("Amount payable: Rs.", round(amount, 2))

# Operator precedence
print("\n--- Expression Examples ---")
print("(4.2 - 3.5) / 0.1 =", (4.2 - 3.5) / 0.1)
print("round((4.2 - 3.5) / 0.1) =", round((4.2 - 3.5) / 0.1))
```

```
32
33  # Difference between round(), int(), and floor()
34  x = -3.7
35  print("\nFor x = -3.7:")
36  print("round(x) =", round(x))
37  print("int(x) =", int(x))
38  print("math.floor(x) =", math.floor(x))
39
40  # Comparison chaining and Boolean operators
41  print("\n--- Validation ---")
42  print("2.0 <= fat <= 7.0:", 2.0 <= fat <= 7.0)
43  print("Fat and SNF valid:", 2.0 <= fat <= 7.0 and snf >= 0)
44
45  # Three surprising results
46  print("\n--- Unexpected Results ---")
47  print("0.1 + 0.2 =", 0.1 + 0.2)
48  print("0.1 + 0.2 == 0.3:", 0.1 + 0.2 == 0.3)
49  print("round(2.5) =", round(2.5))
50  print("int(-3.7) =", int(-3.7))
```

OTPUT

```
Enter fat percentage: 4.2
Enter SNF percentage: 8.5
Enter milk quantity in litres: 10
```

INPUT

```
--- Payment Calculation ---
Fat steps: 12
SNF steps: 5
Rate per litre: Rs. 36.3
Amount payable: Rs. 363.0

--- Expression Examples ---
(4.2 - 3.5) / 0.1 = 7.000000000000002
round((4.2 - 3.5) / 0.1) = 7

For x = -3.7:
round(x) = -4
int(x) = -3
math.floor(x) = -4

--- Validation ---
2.0 <= fat <= 7.0: True
Fat and SNF valid: True

--- Unexpected Results ---
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3: False
round(2.5) = 2
int(-3.7) = -3
```

TASK 5 Read Input and Convert Types

Aim:

To design an interactive data ingestion interface that reads standard terminal inputs, implements robust validation and exception-handling routines against malformed entries, and produces clean, aligned reports using f-string formatting syntax.

Theory:

The input() function reads data from the user as text. Numeric entries must be converted using functions such as int() or float(). Invalid entries should be handled with try and except so the program does not stop unexpectedly. The system should validate milk quantity, fat, and SNF before calculating the payment. Extra spaces and incorrect characters should be handled carefully. F-strings can display amounts and totals in a clear, aligned format.

PROGRAM

```
def read_number(prompt):
    while True:
        text = input(prompt).strip()

        # Accept decimal point or decimal comma
        text = text.replace(",", ".")

        try:
            return float(text)
        except ValueError:
            print("Invalid input! Please enter a valid number.")

# Read clerk's entries
farmer_id = input("Enter Farmer ID: ")
litres = read_number("Enter milk quantity in litres: ")
fat = read_number("Enter fat percentage: ")
snf = read_number("Enter SNF percentage: ")

# Validate values
if litres <= 0:
    print("Error: Milk quantity must be greater than zero.")
elif not (2.0 <= fat <= 7.0):
    print("Error: Fat must be between 2.0 and 7.0.")
elif not (0 <= snf <= 100):
    print("Error: SNF must be between 0 and 100.")
else:
    # Example payment formula
    rate = 30.0 + fat * 0.40 + snf * 0.30
    amount = litres * rate

    print("\n--- Farmer Payment Receipt ---")
```

–

```
print(f"{'Farmer ID:':<22}{farmer_id}")
print(f"{'Milk quantity:':<22}{litres:>8.2f} litres")
print(f"{'Fat percentage:':<22}{fat:>8.2f}%")
print(f"{'SNF percentage:':<22}{snf:>8.2f}%")
print(f"{'Rate per litre:':<22}Rs. {rate:>8.2f}")
print(f"{'Amount payable:':<22}Rs. {amount:>8.2f}")
```

OTPUT

```
Enter Farmer ID: F101
Enter milk quantity in litres: 10
Enter fat percentage: 5
Enter SNF percentage: 5,8
```

INPUT

```
--- Farmer Payment Receipt ---
Farmer ID:            F101
Milk quantity:           10.00 litres
Fat percentage:           5.00%
SNF percentage:           5.80%
Rate per litre:       Rs.    33.74
Amount payable:       Rs.   337.40
PS C:\Users\venkata sai mohith\OneDrive\Documents\co1_assessement 1>
```

TASK 6 Organise the Logic into Functions

Aim:

To decompose the monolithic billing calculation into modular, testable functional components, analyze stack frame mechanics and namespace scoping under the LEGB hierarchy, and document functional contracts using standard docstrings.

Theory:

Functions divide a large program into smaller, reusable parts. In a milk billing program, separate functions can calculate the rate per litre, payment, and ten-day total. Each function should have one clear responsibility. Returning a value makes a function easier to test and reuse than printing results directly. Docstrings explain what a function does, its inputs, and its output. Modular code is easier to maintain, test, and improve.

PROGRAM

```python
# Global constants
BASE_RATE = 30.0
FAT_RATE = 0.40
SNF_RATE = 0.30
DAYS = 10


def rate_per_litre(fat, snf):
    """Calculate and return the milk rate per litre."""
    rate = BASE_RATE + fat * FAT_RATE + snf * SNF_RATE
    return rate


def payout(litres, rate):
    """Calculate and return the payment for the milk delivered."""
    return litres * rate


def ten_day_total(daily_amount, days=DAYS):
    """Calculate and return the total for the given number of days."""
    return daily_amount * days


def display_receipt(farmer_id, litres, fat, snf, rate, amount, total):
    """Print the farmer's payment details."""
    print("\n--- Farmer Payment Receipt ---")
    print(f"Farmer ID:          {farmer_id}")
    print(f"Milk quantity:      {litres:.2f} litres")
    print(f"Fat percentage:     {fat:.2f}%")
    print(f"SNF percentage:     {snf:.2f}%")
    print(f"Rate per litre:     Rs. {rate:.2f}")
```

–

```
32      print(f"Daily payment:      Rs. {amount:.2f}")
33      print(f"Ten-day total:      Rs. {total:.2f}")
34
35
36  # Main program
37  farmer_id = input("Enter Farmer ID: ")
38  litres = float(input("Enter milk quantity in litres: "))
39  fat = float(input("Enter fat percentage: "))
40  snf = float(input("Enter SNF percentage: "))
41
42  if litres <= 0 or not (2.0 <= fat <= 7.0) or not (0 <= snf <= 100):
43      print("Invalid input. Check litres, fat and SNF values.")
44  else:
45      rate = rate_per_litre(fat, snf)
46      amount = payout(litres, rate)
47      total = ten_day_total(amount)
48
49      display_receipt(
50          farmer_id, litres, fat, snf, rate, amount, total
51      )
```

OUTPUT

```
Enter Farmer ID: F101
Enter milk quantity in litres: 10
Enter fat percentage: 5
Enter SNF percentage: 8.5
```

INPUT

```
--- Farmer Payment Receipt ---
Farmer ID:          F101
Milk quantity:      10.00 litres
Fat percentage:     5.00%
SNF percentage:     8.50%
Rate per litre:     Rs. 34.55
Daily payment:      Rs. 345.50
Ten-day total:      Rs. 3455.00
```

TASK 7 Build a Module

Aim:

To encapsulate domain pricing rules and functions into an independent module (milkrate.py), implement script execution guards using __name__ == "__main__", examine module import resolution paths, and analyze compilation caching mechanisms.

Theory:

A Python module stores related functions and constants in a separate file. For example, milkrate.py can contain the milk-rate and payment calculations. Other programs can import the module and reuse its functions. The condition if __name__ == "__main__" allows test code to run only when the file is executed directly. Python searches module locations when importing a file. Imported modules are cached, which helps avoid loading the same module repeatedly in one process.

PROGRAM

–

```python
    Rate card of the village milk co-operative (used as a module).

BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30


def rate_per_litre(fat, snf):
    """Return the price per litre, rounded to 2 decimal places."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = (
        BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE
    )
    return round(rate, 2)


def payout(litres, fat, snf):
    """Return the farmer's payment, rounded to 2 decimal places."""
    amount = litres * rate_per_litre(fat, snf)
    return round(amount, 2)


if __name__ == "__main__":
    for litres, fat, snf in [
        (10, 3.5, 8.5),
        (8.5, 4.2, 8.7),
        (12, 3.1, 8.3)
    ]:
        (variable) pay: Any      t, snf)
        pay = payout(litres, fat, snf)
        print(
            f"{litres:g} L fat {fat:g} snf {snf:g} "
            f"rate {rate:.2f} pay {pay:.2f}"
        )

```

INPUT

```
10 L fat 3.5 snf 8.5 rate 30.00 pay 300.00
8.5 L fat 4.2 snf 8.7 rate 33.40 pay 283.90
12 L fat 3.1 snf 8.3 rate 27.80 pay 333.60
```

TASK 8 Plan the Algorithm Before Coding

Aim:

To formulate an algorithmic blueprint for ten-day payment calculations, design structured validation logic, develop pseudocode conforming to engineering standards, execute manual hand traces, and compare streaming accumulation with buffered list storage.

Theory:

Planning the algorithm before coding helps organize the solution. Pseudocode and flowcharts describe the steps in a clear, language-independent way. The program should first read and validate the milk details, then calculate the payment. A trace table can be used to check how values change during each step. A running total uses little extra memory when only the final total is required. A list is useful when individual transaction records must also be stored or printed.

```python
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
DAYS = 10

def calculate_payment(litres, fat, snf):
    """Return the rate per litre and payment for one entry."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE
    rate += snf_steps * SNF_STEP_VALUE
    rate = round(rate, 2)

    amount = round(litres * rate, 2)
    return rate, amount

farmer_id = input("Enter Farmer ID: ")
total = 0.00
for day in range(1, DAYS + 1):
    print(f"\nDay {day}")

    try:
        litres = float(input("Enter litres: "))
        fat = float(input("Enter fat percentage: "))
        snf = float(input("Enter SNF percentage: "))
        shift = input("Enter shift (Morning/Evening): ").strip().title()

        if (litres <= 0 or not 2.0 <= fat <= 7.0
                or not 0 <= snf <= 100
                or shift not in ("Morning", "Evening")):
```

```
            print("Invalid entry. Please enter this day again.")
            # Retry the same day
            while True:
                try:
                    litres = float(input("Enter litres: "))
                    fat = float(input("Enter fat percentage: "))
                    snf = float(input("Enter SNF percentage: "))
                    shift = input(
                        "Enter shift (Morning/Evening): "
                    ).strip().title()

                    if (litres > 0 and 2.0 <= fat <= 7.0
                            and 0 <= snf <= 100
                            and shift in ("Morning", "Evening")):
                        break
                    print("Invalid entry. Try again.")
                except ValueError:
                    print("Enter numeric values correctly.")
        rate, amount = calculate_payment(litres, fat, snf)
    except ValueError:
        print("Invalid numeric input. Please restart the program.")
        break
    total = round(total + amount, 2)
    print(f"Rate per litre: Rs. {rate:.2f}")
    print(f"Daily payment: Rs. {amount:.2f}")
    print(f"Running total: Rs. {total:.2f}")
else:
    print("\n--- Ten-Day Payment Summary ---")
    print("Farmer ID:", farmer_id)
    print(f"Total payment: Rs. {total:.2f}")
```

INPUT AND OUTPUT

```
Day 1
Enter litres: 10
Enter fat percentage: 3.5
Enter SNF percentage: 8.5
Enter shift (Morning/Evening): Morning
Rate per litre: Rs. 30.00
Daily payment: Rs. 300.00
Running total: Rs. 300.00

Day 2
Enter litres: 8.5
Enter fat percentage: 4.2
Enter SNF percentage: 8.7
Enter shift (Morning/Evening): Evening
Rate per litre: Rs. 33.40
Daily payment: Rs. 283.90
Running total: Rs. 583.90

Day 3
Enter litres: 12
Enter fat percentage: 3.1
Enter SNF percentage: 8.3
Enter shift (Morning/Evening): Morning
Rate per litre: Rs. 27.80
Daily payment: Rs. 333.60
Running total: Rs. 917.50
```

–

TASK 9 Implement and Test the Calculator

Aim:

To implement the complete, end-to-end payment calculator, construct a multi-category test harness covering nominal, boundary, and error conditions, and demonstrate the efficiency of automated regression tests over manual verification.

Theory:

Testing checks whether the billing program produces correct results. Tests should include normal values, values at the limits, and invalid inputs. The standard fat and SNF values should also be tested to confirm the base-rate calculation. Boundary testing helps find errors that may occur near the minimum and maximum allowed values. Automated assertions make repeated testing faster and more consistent. Clear error messages help prevent invalid records from entering the payment ledger.

PROGRAM

```python
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
MIN_FAT = 2.0
MAX_FAT = 7.0
def calculate_payment(litres, fat, snf):
    """Calculate the milk rate and payment after validating inputs."""

    if litres <= 0:
        raise ValueError("Litres must be greater than zero")

    if not MIN_FAT <= fat <= MAX_FAT:
        raise ValueError("Fat must be between 2.0 and 7.0")

    if not 0 <= snf <= 100:
        raise ValueError("SNF must be between 0 and 100")

    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)

    rate = (
        BASE_RATE
        + fat_steps * FAT_STEP_VALUE
        + snf_steps * SNF_STEP_VALUE
    )

    rate = round(rate, 2)
    amount = round(litres * rate, 2)

    return rate, amount

```

```python
def run_tests():
    """Test valid and invalid payment entries."""

    valid_tests = [
        ("Standard", 10, 3.5, 8.5, 30.00, 300.00),
        ("Above standard", 10, 4.0, 8.7, 32.60, 326.00),
        ("Below standard", 10, 3.0, 8.3, 27.40, 274.00),
        ("Minimum fat", 10, 2.0, 8.5, 24.00, 240.00),
        ("Maximum fat", 10, 7.0, 8.5, 44.00, 440.00),
    ]

    print("--- VALID TESTS ---")

    for name, litres, fat, snf, expected_rate, expected_amount in valid_tests:
        rate, amount = calculate_payment(litres, fat, snf)

        assert rate == expected_rate, f"{name}: incorrect rate"
        assert amount == expected_amount, f"{name}: incorrect payment"

        print(
            f"{name}: Expected = Rs. {expected_amount:.2f}, "
            f"Actual = Rs. {amount:.2f} : PASS"
        )

    invalid_tests = [
        ("Negative litres", -1, 3.5, 8.5),
        ("Zero litres", 0, 3.5, 8.5),
        ("Fat below minimum", 10, 1.9, 8.5),
        ("Fat above maximum", 10, 7.1, 8.5),
    ]

    print("\n--- INVALID TESTS ---")

    for name, litres, fat, snf in invalid_tests:
        try:
            calculate_payment(litres, fat, snf)
        except ValueError:
            print(f"{name}: Correctly rejected - PASS")
        else:
            raise AssertionError(f"{name}: should have been rejected")

    # Test text conversion
    print("\n--- TEXT INPUT TEST ---")

    try:
        float("eight")
    except ValueError:
        print("Text 'eight': Correctly rejected - PASS")

    print("\nAll tests passed successfully!")


if __name__ == "__main__":
    run_tests()
```

Output

```
--- VALID TESTS ---
Standard: Expected = Rs. 300.00, Actual = Rs. 300.00 : PASS
Above standard: Expected = Rs. 326.00, Actual = Rs. 326.00 : PASS
Below standard: Expected = Rs. 274.00, Actual = Rs. 274.00 : PASS
Minimum fat: Expected = Rs. 240.00, Actual = Rs. 240.00 : PASS
Maximum fat: Expected = Rs. 440.00, Actual = Rs. 440.00 : PASS

--- INVALID TESTS ---
Negative litres: Correctly rejected - PASS
Zero litres: Correctly rejected - PASS
Fat below minimum: Correctly rejected - PASS
Fat above maximum: Correctly rejected - PASS
Negative SNF: Correctly rejected - PASS

--- TEXT INPUT TEST ---
Text 'eight': Correctly rejected - PASS
```

TASK 10 – Compare Expectations with Execution

Aim:

To benchmark runtime performance and numerical drift between floating-point and Decimal data types across scaling batch sizes, evaluate execution speeds using time.perf_counter(), analyze the impact of line-item versus batch rounding, and assess lookup-table optimizations.

Theory:

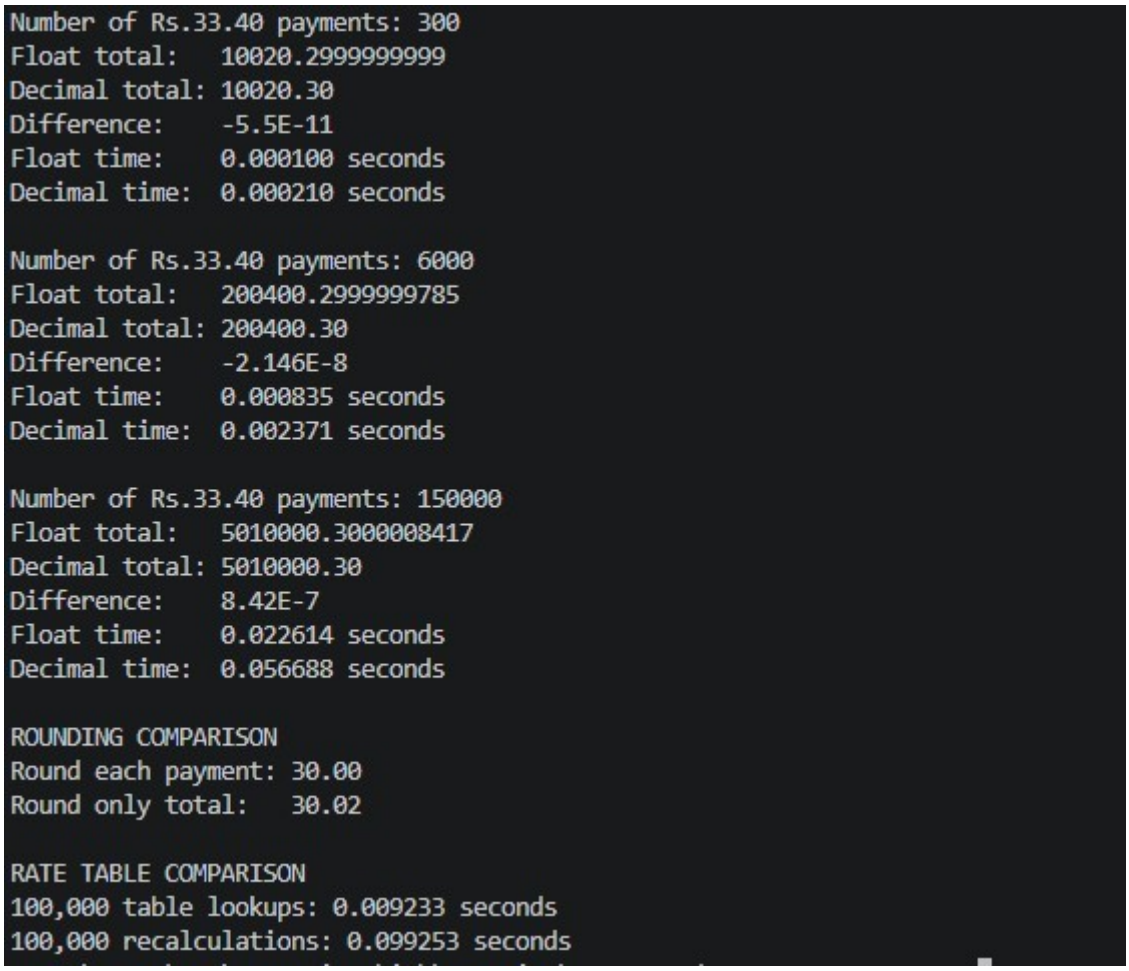
Performance testing compares the speed and accuracy of different calculation methods. Python's time.perf_counter() can measure execution time. Floating-point calculations may be fast, but their small representation errors can accumulate in large batches. Decimal arithmetic is often more suitable when accurate financial amounts are required. The rounding policy should specify whether each payment or only the final total is rounded. A lookup table can also reduce repeated calculations when many records use the same rates.

PROGRAM

```python
from decimal import Decimal
import time
PAYMENT = "33.40"
ADJUSTMENT = "0.10"
SIZES = [300, 6000, 150000]
def total_float(payments):
    """Calculate total using floating-point arithmetic."""
    total = 0.0
    for amount in payments:
        total += float(amount)
    return total
def total_decimal(payments):
    """Calculate exact decimal total from string amounts."""
    total = Decimal("0.00")
    for amount in payments:
        total += Decimal(amount)
    return total
def measure(function, payments):
    """Return the result and elapsed time."""
    start = time.perf_counter()
    result = function(payments)
    elapsed = time.perf_counter() - start
    return result, elapsed
print("FLOAT vs DECIMAL TOTALS")
print("-" * 75)
for n in SIZES:
    payments = [PAYMENT] * n + [ADJUSTMENT] * 3
    float_total, float_time = measure(total_float, payments)
    decimal_total, decimal_time = measure(total_decimal, payments)
    difference = Decimal(str(float_total)) - decimal_total
    print(f"\nNumber of Rs.33.40 payments: {n}")
```

```
    print(f"Float total:   {float_total:.10f}")
    print(f"Decimal total: {decimal_total:.2f}")
    print(f"Difference:    {difference}")
    print(f"Float time:    {float_time:.6f} seconds")
    print(f"Decimal time:  {decimal_time:.6f} seconds")
# Compare rounding strategies
print("\nROUNDING COMPARISON")
amounts = [Decimal("10.005"), Decimal("10.005"), Decimal("10.005")]
round_each = sum(
    (amount.quantize(Decimal("0.01")) for amount in amounts),
    Decimal("0.00")
)
round_total = sum(amounts, Decimal("0.00")).quantize(
    Decimal("0.01")
)
print("Round each payment:", round_each)
print("Round only total:  ", round_total)
# Rate-table lookup versus recalculation
print("\nRATE TABLE COMPARISON")
RATE_CARD = {
    (35, 85): Decimal("30.00"),
    (42, 87): Decimal("33.40"),
    (31, 83): Decimal("27.80")
}
start = time.perf_counter()
for _ in range(100000):
    rate = RATE_CARD[(42, 87)]
lookup_time = time.perf_counter() - start
start = time.perf_counter()
for _ in range(100000):
    rate = Decimal("30.00") + Decimal("12") * Decimal("0.40")
recalculation_time = time.perf_counter() - start
print(f"100,000 table lookups: {lookup_time:.6f} seconds")
print(f"100,000 recalculations: {recalculation_time:.6f} seconds")
```

OUTPUT


Number of Rs.33.40 payments: 300
Float total: 10020.2999999999
Decimal total: 10020.30
Difference: -5.5E-11
Float time: 0.000100 seconds
Decimal time: 0.000210 seconds
Number of Rs.33.40 payments: 6000
Float total: 200400.2999999785
Decimal total: 200400.30
Difference: -2.146E-8
Float time: 0.000835 seconds
Decimal time: 0.002371 seconds
Number of Rs.33.40 payments: 150000
Float total: 5010000.3000008417
Decimal total: 5010000.30
Difference: 8.42E-7
Float time: 0.022614 seconds
Decimal time: 0.056688 seconds
ROUNDING COMPARISON
Round each payment: 30.00
Round only total: 30.02
RATE TABLE COMPARISON
100,000 table lookups: 0.009233 seconds
100,000 recalculations: 0.099253 seconds

TASK 11 – Recommend and Defend the Final Design

Aim:

To synthesize empirical findings into a production-ready system recommendation, justify the choice of numeric data types and configuration models, evaluate system constraints, and defend architectural scalability up to district-union operations.

Theory:

A reliable milk billing system should balance accuracy, speed, and ease of maintenance. Decimal arithmetic and a clearly defined rounding policy help protect payment accuracy. Small systems may store rates in code, while larger systems can keep them in a configuration file. Separating rates from program logic makes authorized price updates easier. Large collection centers may need a database and batch processing to manage many records. These design choices help the system grow while keeping calculations consistent.

PROGRAM

```
1
2   import time
3
4   start = time.perf_counter()
5
6   # Co-operative product rates
7   rates = {
8       "Milk": 42.567,
9       "Rice": 55.234,
10      "Sugar": 38.999,
11      "Oil": 120.456
12  }
13
14  print("CO-OPERATIVE RATE CALCULATOR")
15  print("----------------------------")
16
17  for item, rate in rates.items():
18      final_rate = round(rate, 2)
19      print(item, "Original Rate:", rate,
20            "Final Rate:", final_rate)
21
22  end = time.perf_counter()
23
24  print("\nStart-up and calculation time:",
25        round(end - start, 6), "seconds")
26
27  print("\nRecommendation:")
28  print("Round rates to 2 decimal places.")
29  print("Update rates when prices change.")
30  print("This program is suitable for small co-operatives.")
31
```

Output

```
CO-OPERATIVE RATE CALCULATOR
----------------------------
Milk Original Rate: 42.567 Final Rate: 42.57
Rice Original Rate: 55.234 Final Rate: 55.23
Sugar Original Rate: 38.999 Final Rate: 39.0
Oil Original Rate: 120.456 Final Rate: 120.46

Start-up and calculation time: 0.016982 seconds
```

TASK 12 – Re-evaluate and Adapt the Design

Aim:

To re-evaluate the system architecture against evolving business requirements, adapt calculations for high-resolution two-decimal analyzer inputs, implement seasonal rate switching and tiered volume bonuses, and defend the updated design using test suites.

Theory:

The billing system should be adaptable when business requirements change. Modern analyzers may provide fat and SNF values with two decimal places, so calculations must preserve fractional quality steps. Simply converting these values to integers may cause incorrect payments. Configurable rate tables support seasonal prices and different quality incentives. Volume bonuses should be calculated at the end of the billing period using the total quantity supplied. Decimal arithmetic and updated tests help maintain accuracy as new features are added.

Program

```python
from decimal import Decimal, ROUND_HALF_UP

# Seasonal rate settings
BASE_RATE = Decimal("32.00")
FAT_STEP_RATE = Decimal("0.45")
BONUS_RATE = Decimal("0.50")
BONUS_LIMIT = 500


def get_rate_card(season):
    if season.lower() in ["april", "may", "june"]:
        return BASE_RATE, FAT_STEP_RATE
    else:
        return Decimal("30.00"), Decimal("0.40")


def calculate_rate(fat, snf, season, litres):
    base, step_rate = get_rate_card(season)

    # Example: each 1.00 fat unit above 3.00 earns one step
    fat_steps = max(0, int((fat - Decimal("3.00")).to_integral_value(
        rounding=ROUND_HALF_UP
    )))

    rate = base + Decimal(fat_steps) * step_rate

    if litres > BONUS_LIMIT:
        rate += BONUS_RATE

    return rate.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
```

```
32
33
34  # Input
35  season = input("Enter season/month: ").strip()
36  fat = Decimal(input("Enter fat value (e.g. 4.25): "))
37  snf = Decimal(input("Enter SNF value (e.g. 8.50): "))
38  litres = Decimal(input("Enter total litres supplied in 10 days: "))
39
40  # Calculation
41  final_rate = calculate_rate(fat, snf, season, litres)
42
43  print("\n--- MILK RATE CALCULATOR ---")
44  print("Season:", season)
45  print("Fat:", fat)
46  print("SNF:", snf)
47  print("Base rate: Rs.", get_rate_card(season)[0])
48  print("Bonus eligible:", "Yes" if litres > BONUS_LIMIT else "No")
49  print("Final rate per litre: Rs.", final_rate)
50  print("Total payment: Rs.", final_rate * litres)
51
```

Output

```
Enter season/month: april
Enter fat value (e.g. 4.25): 4.25
Enter SNF value (e.g. 8.50): 8.50
Enter total litres supplied in 10 days: 520

--- MILK RATE CALCULATOR ---
Season: april
Fat: 4.25
SNF: 8.50
Base rate: Rs. 32.00
Bonus eligible: Yes
Final rate per litre: Rs. 32.95
Total payment: Rs. 17134.00
```